Proof Verification Summary

Mathematical Statement:

a prime number is a natural number greater than 1 that is not a product of two smaller natural numbers

Total Iterations:

24

Result:

A valid proof has been found!

Latest Iteration

Latest Proof:

```
module temp_proof where

data Bool : Set where
true : Bool
false : Bool

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

ifZero : ℕ → Bool
```

```
ifZero zero = true
ifZero (suc _) = false

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

proj₁ : ∀ {A B : Set} → A × B → A
proj₁ (a , b) = a

proj₂ : ∀ {A B : Set} → A × B → B
proj₂ (a , b) = b

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper (suc n) m with n ... | true = zero , suc n
... | false = let qr = divideHelper n m in suc (proj₁ qr) , proj₂ qr

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | qr with ifZero (proj₂ qr)
... | true = true
... | false = false

hasProperFactor : ℕ → ℕ → Bool
hasProperFactor n zero = false
hasProperFactor n (suc zero) = false
hasProperFactor n (suc (suc m)) =
((suc (suc m)
_>_ : ℕ → ℕ → Bool
m > n = n
isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasProperFactor n (n - suc zero))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Latest Verification Result:

Proof is valid.

Feedback:
Proof has been compiled and verified after 24 iterations.

Iteration 1

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x
```

```
data Bool : Set where
true : Bool
false : Bool

≤ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasFactor n (suc (suc zero)))

hasFactor : ℕ → ℕ → Bool
hasFactor n m with m ... | true = divides m n ∨ hasFactor n (suc m)
... | false = false

divides : ℕ → ℕ → Bool
divides m n with n ... | true = false
divides m n with n ≡ (m * (divide n m))
... | true = true
... | false = false

divide : ℕ → ℕ → ℕ
divide zero _ = zero
divide n m with n ... | true = zero
divide (suc n) m = suc (divide (n - m) m)

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:23,1-9
Could not parse the left-hand side zero ≤ _
Operators used in the grammar:
None
when scope checking the left-hand side zero ≤ _ in the definition
of ≤

Iteration 2

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

data Bool : Set where
true : Bool
false : Bool

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasFactor n (suc (suc zero)))

hasFactor : ℕ → ℕ → Bool
hasFactor n m with m ... | true = divides m n ∨ hasFactor n (suc m)
... | false = false

divides : ℕ → ℕ → Bool
divides m n with n ... | true = false
divides m n with m * (divide n m) ≡ n
... | true = true
... | false = false

divide : ℕ → ℕ → ℕ
divide zero _ = zero
divide n m with n ... | true = zero
divide (suc n) m = suc (divide (n - m) m)

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
```

primeExample = mkPrimeProof (suc (suc zero)) refl

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:28,1-6
Could not parse the left-hand side m Problematic expression: ((m Operators used in the grammar:
when scope checking the left-hand side m _

Iteration 3

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasFactor n (suc (suc zero)))

hasFactor : ℕ → ℕ → Bool
hasFactor n m with m ... | true = divides m n ∨ hasFactor n (suc m)
... | false = false

divides : ℕ → ℕ → Bool
divides m n with m * (divide n m) ≡ n
... | true = true
... | false = false

divide : ℕ → ℕ → ℕ
```

```
divide zero _ = zero
divide n m with n ... | true = zero
divide (suc n) m = suc (divide (n - m) m)

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:18,15-19
Not in scope:
Bool
at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:18,15-19
when scope checking Bool

Iteration 4

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

data Bool : Set where
true : Bool
false : Bool

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
```

```
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasFactor n (suc (suc zero)))

hasFactor : ℕ → ℕ → Bool
hasFactor n m with m ... | true = divides m n ∨ hasFactor n (suc m)
... | false = false

divides : ℕ → ℕ → Bool
divides m n = (m * (divide n m)) ≡ n

divide : ℕ → ℕ → ℕ
divide zero _ = zero
divide n m with n ... | true = zero
divide (suc n) m = suc (divide (n - m) m)

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

```
Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:28,1-6
Could not parse the left-hand side m Problematic expression: ((m Operators used in the grammar:
when scope checking the left-hand side m _
```

Iteration 5

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

data Bool : Set where
true : Bool
false : Bool
```

```
_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasFactor n (suc (suc zero)))

hasFactor : ℕ → ℕ → Bool
hasFactor n m with m ... | true = divides m n ∨ hasFactor n (suc m)
... | false = false

divides : ℕ → ℕ → Bool
divides m n = ifHelper (m * (divide n m)) n

ifHelper : ∀ x y → (x ≡ y)
ifHelper {x = zero} {y = zero} = refl
ifHelper {x = suc x} {y = suc y} with ifHelper {x} {y}
... | refl = refl

divide : ℕ → ℕ → ℕ
divide zero _ = zero
divide n m with n ... | true = zero
divide (suc n) m = suc (divide (n - m) m)

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:46,33-42
Not in scope:
hasFactor
at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:46,33-42
when scope checking hasFactor

Iteration 6

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

data Bool : Set where
true : Bool
false : Bool

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasFactor n (suc (suc zero)))

hasFactor : ℕ → ℕ → Bool
hasFactor n m with m ... | true = divides m n ∨ hasFactor n (suc m)
... | false = false

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) = ifHelper (m * q) n

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper n m with n ... | true = zero , n
divideHelper n m with divideHelper (n - m) m
... | (q , r) = suc q , r

ifHelper : ∀ x y → (x ≡ y)
ifHelper {x = zero} {y = zero} = refl
ifHelper {x = suc x} {y = suc y} with ifHelper {x} {y}
... | refl = refl

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m
```

```
data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

```
Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:46,33-42
Not in scope:
hasFactor
at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:46,33-42
when scope checking hasFactor
```

Iteration 7

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

data Bool : Set where
true : Bool
false : Bool

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasFactor n (suc (suc zero)))
```

```
hasFactor : ℕ → ℕ → Bool
hasFactor n m with m ... | true = divides m n ∨ hasFactor n (suc m)
... | false = false

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) with ifZero r
... | true = true
... | false = false

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper n m with n ... | true = zero , n
divideHelper n m with divideHelper (n - m) m
... | (q , r) = suc q , r

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

```
Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:46,33-42
Not in scope:
hasFactor
at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:46,33-42
when scope checking hasFactor
```

Iteration 8

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

data Bool : Set where
true : Bool
```

```
false : Bool

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper n m with n ... | true = zero , n
divideHelper n m with divideHelper (n - m) m
... | (q , r) = suc q , r

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) with ifZero r
... | true = true
... | false = false

hasFactor : ℕ → ℕ → Bool
hasFactor n m with m ... | true = divides m n ∨ hasFactor n (suc m)
... | false = false

isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasFactor n (suc (suc zero)))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:54,26-27
Not in scope:
×
at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:54,26-27
when scope checking ×

Iteration 9

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

data Bool : Set where
true : Bool
false : Bool

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper n m with n ... | true = zero , n
divideHelper n m with divideHelper (n - m) m
... | (q , r) = suc q , r

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) with ifZero r
... | true = true
... | false = false
```

```
hasFactor : ℕ → ℕ → Bool
hasFactor n m with m ... | true = divides m n ∨ hasFactor n (suc m)
... | false = false

isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasFactor n (suc (suc zero)))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:54,26-27
Not in scope:
×
at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:54,26-27
when scope checking ×

Iteration 10

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

data Bool : Set where
true : Bool
false : Bool

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false
```

```
_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper n m with n ... | true = zero , n
divideHelper n m with divideHelper (n - m) m
... | (q , r) = suc q , r

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) with ifZero r
... | true = true
... | false = false

hasFactor : ℕ → ℕ → Bool
hasFactor n m with m ... | true = divides m n ∨ hasFactor n (suc m)
... | false = false

isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasFactor n (suc (suc zero)))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

```
Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:60,1-11
Incomplete pattern matching for temp_proof.with-98. Missing cases:
divideHelper n m | false
when checking the definition of temp_proof.with-98
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:61,1-62,26
warning: -W[no]UnreachableClauses
Unreachable clause
when checking the definition of divideHelper
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:57,1-62,26
Termination checking failed for the following functions:
divideHelper
Problematic calls:
divideHelper (n - m) m
(at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:61,23-35)
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:70,1-73,20
Termination checking failed for the following functions:
hasFactor
Problematic calls:
hasFactor n m | suc m ≤ n
hasFactor n (suc m)
(at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:72,28-37)
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:82,46-50
```

(¬ (hasFactor (suc (suc zero)) (suc (suc zero)))) != true of type
Bool
when checking that the expression refl has type
isPrime (suc (suc zero)) ≡ true

Iteration 11

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

data Bool : Set where
true : Bool
false : Bool

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper n m with n ... | true = zero , n
divideHelper n m with divideHelper (n - m) m
```

... | (q , r) with n ... | true = suc q , r
... | false = zero , m

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) with ifZero r
... | true = true
... | false = false

hasFactor : ℕ → ℕ → Bool
hasFactor n m with m ... | true = divides m n ∨ hasFactor n (suc m)
... | false = false

isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasFactor n (suc (suc zero)))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:60,1-11
Incomplete pattern matching for temp_proof.with-98. Missing cases:
divideHelper n m | false
when checking the definition of temp_proof.with-98
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:61,1-64,25
warning: -W[no]UnreachableClauses
Unreachable clause
when checking the definition of divideHelper
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:57,1-64,25
Termination checking failed for the following functions:
divideHelper
Problematic calls:
divideHelper (n - m) m
(at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:61,23-35)
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:72,1-75,20
Termination checking failed for the following functions:
hasFactor
Problematic calls:
hasFactor n m | suc m ≤ n
hasFactor n (suc m)
(at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:74,28-37)
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:84,46-50
(¬ (hasFactor (suc (suc zero)) (suc (suc zero)))) != true of type
Bool
when checking that the expression refl has type
isPrime (suc (suc zero)) ≡ true

Iteration 12

Proof:

module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ

```
zero + n = n
suc m + n = suc (m + n)

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

data Bool : Set where
true : Bool
false : Bool

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper n m with n ... | true = zero , n
divideHelper n m with divideHelper (n - m) m
... | (q , r) = suc q , r

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) with ifZero r
... | true = true
... | false = false

hasProperFactor : ℕ → ℕ → Bool
hasProperFactor n m with m ... | true = divides m n ∨ hasProperFactor n (suc m)
... | false = false

isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasProperFactor n (suc zero))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:60,1-11
Incomplete pattern matching for temp_proof.with-98. Missing cases:
divideHelper n m | false
when checking the definition of temp_proof.with-98
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:61,1-62,26
warning: -W[no]UnreachableClauses
Unreachable clause
when checking the definition of divideHelper
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:57,1-62,26
Termination checking failed for the following functions:
divideHelper
Problematic calls:
divideHelper (n - m) m
(at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:61,23-35)
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:70,1-73,20
Termination checking failed for the following functions:
hasProperFactor
Problematic calls:
hasProperFactor n m | suc m ≤ n
hasProperFactor n (suc m)
(at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:72,28-43)
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:82,46-50
(¬ (hasProperFactor (suc (suc zero)) (suc zero))) != true of type
Bool
when checking that the expression refl has type
isPrime (suc (suc zero)) ≡ true

Iteration 13

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

data Bool : Set where
true : Bool
false : Bool

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
```

```
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper n m with n ... | true = zero , n
divideHelper n m with n - m
... | k with divideHelper k m
... | (q , r) = suc q , r

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) with ifZero r
... | true = true
... | false = false

hasProperFactor : ℕ → ℕ → Bool
hasProperFactor n zero = false
hasProperFactor n (suc zero) = false
hasProperFactor n (suc (suc m)) with m + suc (suc m) ... | true = divides (suc (suc m)) n ∨ hasProperFactor n (suc m)
... | false = false

isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasProperFactor n (suc n))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:74,38-57
Could not parse the application m + suc (suc m) Operators used in the grammar:
+ (infix operator, level 20) [_+_ (D:\Daniel\Braude\Year 4\Semester
G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:7,1-4)]
when scope checking m + suc (suc m)

Iteration 14

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

data Bool : Set where
true : Bool
false : Bool

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper n m with n ... | true = zero , n
divideHelper n m with n - m
... | k with divideHelper k m
... | (q , r) = suc q , r

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) with ifZero r
... | true = true
... | false = false

hasProperFactor : ℕ → ℕ → Bool
hasProperFactor n zero = false
```

```
hasProperFactor n (suc zero) = false
hasProperFactor n (suc (suc m)) =
let k = suc (suc m) in
(k
isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasProperFactor n (n - suc zero))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

```
Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:76,4-23
Could not parse the application k Operators used in the grammar:
∧ (infix operator, level 20) [_∧_ (D:\Daniel\Braude\Year 4\Semester
G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:37,1-4)]
when scope checking k
```

Iteration 15

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

data Bool : Set where
true : Bool
false : Bool

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false
```

```
_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper n m with n ... | true = zero , n
divideHelper n m with n - m
... | k with divideHelper k m
... | (q , r) = suc q , r

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) with ifZero r
... | true = true
... | false = false

hasProperFactor : ℕ → ℕ → Bool
hasProperFactor n zero = false
hasProperFactor n (suc zero) = false
hasProperFactor n (suc (suc m)) =
(suc (suc m)
isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasProperFactor n (n - suc zero))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:75,4-45
Could not parse the application
suc (suc m) Operators used in the grammar:
∧ (infix operator, level 20) [_∧_ (D:\Daniel\Braude\Year 4\Semester
G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:37,1-4)]
when scope checking suc (suc m)

Iteration 16

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ
```

```
_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

data Bool : Set where
true : Bool
false : Bool

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper n m with n ... | true = zero , n
divideHelper n m with n - m
... | k with divideHelper k m
... | (q , r) = suc q , r

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) with ifZero r
... | true = true
... | false = false

hasProperFactor : ℕ → ℕ → Bool
hasProperFactor n zero = false
hasProperFactor n (suc zero) = false
hasProperFactor n (suc (suc m)) =
((_
isPrime : ℕ → Bool
isPrime n = ((n > suc zero) ∧ ¬ (hasProperFactor n (n - suc zero)))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof
```

```
primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

```
Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:61,1-63,26
warning: -W[no]UnreachableClauses
Unreachable clause
when checking the definition of divideHelper
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:60,1-11
Incomplete pattern matching for temp_proof.with-98. Missing cases:
divideHelper n m | false
when checking the definition of temp_proof.with-98
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:57,1-63,26
Termination checking failed for the following functions:
divideHelper
Problematic calls:
divideHelper n m | n - m
divideHelper k m
(at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:62,14-26)
```

Iteration 17

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
```

```
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper n m with n ... | true = zero , n
divideHelper n m with n - m
... | k = with divideHelper k m
... | (q , r) = suc q , r

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) with ifZero r
... | true = true
... | false = false

hasProperFactor : ℕ → ℕ → Bool
hasProperFactor n zero = false
hasProperFactor n (suc zero) = false
hasProperFactor n (suc (suc m)) =
((_
isPrime : ℕ → Bool
isPrime n = ((n > suc zero) ∧ ¬ (hasProperFactor n (n - suc zero)))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

Feedback:
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:58,11-11
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:58,11: Parse error
with
divideHelper k m
... | (q ,...

Iteration 18

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_-_ : ℕ → ℕ → ℕ
```

```
zero - _ = zero
n - zero = n
suc n - suc m = n - m

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper n m with n ... | true = zero , n
... | false with n - m
... | k = let (q , r) = divideHelper k m in suc q , r

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) with ifZero r
... | true = true
... | false = false

hasProperFactor : ℕ → ℕ → Bool
hasProperFactor n zero = false
hasProperFactor n (suc zero) = false
hasProperFactor n (suc (suc m)) =
((_
isPrime : ℕ → Bool
isPrime n = ((n > suc zero) ∧ ¬ (hasProperFactor n (n - suc zero)))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).

D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:23,15-19
Not in scope:
Bool
at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:23,15-19
when scope checking Bool

Iteration 19

Proof:

```
module temp_proof where

data Bool : Set where
true : Bool
false : Bool

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper n m with n ... | true = zero , n
```

```
... | false with n - m
... | k = let (q , r) = divideHelper k m in suc q , r

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) with ifZero r
... | true = true
... | false = false

hasProperFactor : ℕ → ℕ → Bool
hasProperFactor n zero = false
hasProperFactor n (suc zero) = false
hasProperFactor n (suc (suc m)) =
((_
isPrime : ℕ → Bool
isPrime n = ((n > suc zero) ∧ ¬ (hasProperFactor n (n - suc zero)))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:62,17-43
Expected record pattern
when checking the let binding q , r = divideHelper k m

Iteration 20

Proof:

```
module temp_proof where

data Bool : Set where
true : Bool
false : Bool

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
```

```
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper n m with n ... | true = zero , n
... | false with n - m
... | k = let p = divideHelper k m in suc (proj₁ p) , proj₂ p

proj₁ : ∀ {A B : Set} → A × B → A
proj₁ (a , b) = a

proj₂ : ∀ {A B : Set} → A × B → B
proj₂ (a , b) = b

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) with ifZero r
... | true = true
... | false = false

hasProperFactor : ℕ → ℕ → Bool
hasProperFactor n zero = false
hasProperFactor n (suc zero) = false
hasProperFactor n (suc (suc m)) =
((_
isPrime : ℕ → Bool
isPrime n = ((n > suc zero) ∧ ¬ (hasProperFactor n (n - suc zero)))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:62,46-51
Not in scope:
proj₁
at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:62,46-51
when scope checking proj₁

Iteration 21

Proof:

```
module temp_proof where

data Bool : Set where
true : Bool
false : Bool

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

proj₁ : ∀ {A B : Set} → A × B → A
proj₁ (a , b) = a

proj₂ : ∀ {A B : Set} → A × B → B
proj₂ (a , b) = b

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper n m with n ... | true = zero , n
... | false with n - m
... | k = let p = divideHelper k m in suc (proj₁ p) , proj₂ p
```

```
divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) with ifZero r
... | true = true
... | false = false

hasProperFactor : ℕ → ℕ → Bool
hasProperFactor n zero = false
hasProperFactor n (suc zero) = false
hasProperFactor n (suc (suc m)) =
((_
isPrime : ℕ → Bool
isPrime n = ((n > suc zero) ∧ ¬ (hasProperFactor n (n - suc zero)))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:63,1-68,64
Termination checking failed for the following functions:
divideHelper
Problematic calls:
divideHelper n m | suc n ≤ m
temp_proof.with-118 n m false | n - m
divideHelper k m
(at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:68,21-33)

Iteration 22

Proof:

```
module temp_proof where

data Bool : Set where
true : Bool
false : Bool

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x
```

```
_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

proj₁ : ∀ {A B : Set} → A × B → A
proj₁ (a , b) = a

proj₂ : ∀ {A B : Set} → A × B → B
proj₂ (a , b) = b

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper (suc n) m with n ... | true = zero , suc n
... | false = let (q , r) = divideHelper n m in suc q , r

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | (q , r) with ifZero r
... | true = true
... | false = false

hasProperFactor : ℕ → ℕ → Bool
hasProperFactor n zero = false
hasProperFactor n (suc zero) = false
hasProperFactor n (suc (suc m)) =
((_
isPrime : ℕ → Bool
isPrime n = ((n > suc zero) ∧ ¬ (hasProperFactor n (n - suc zero)))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:67,19-45
Expected record pattern
when checking the let binding q , r = divideHelper n m

Iteration 23

Proof:

```
module temp_proof where

data Bool : Set where
true : Bool
false : Bool

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
_>_ : ℕ → ℕ → Bool
m > n = n
¬ : Bool → Bool
¬ true = false
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

proj₁ : ∀ {A B : Set} → A × B → A
proj₁ (a , b) = a

proj₂ : ∀ {A B : Set} → A × B → B
proj₂ (a , b) = b

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper (suc n) m with n ... | true = zero , suc n
... | false = let qr = divideHelper n m in suc (proj₁ qr) , proj₂ qr
```

```
divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | qr with ifZero (proj₂ qr)
... | true = true
... | false = false

hasProperFactor : ℕ → ℕ → Bool
hasProperFactor n zero = false
hasProperFactor n (suc zero) = false
hasProperFactor n (suc (suc m)) =
((_
isPrime : ℕ → Bool
isPrime n = ((n > suc zero) ∧ ¬ (hasProperFactor n (n - suc zero)))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
Verification failed: The statement was not confirmed as valid by the API.

Iteration 24

Proof:

```
module temp_proof where

data Bool : Set where
true : Bool
false : Bool

data ℕ : Set where
zero : ℕ
suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

_-_ : ℕ → ℕ → ℕ
zero - _ = zero
n - zero = n
suc n - suc m = n - m

_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)

data _≡_ {A : Set} : A → A → Set where
refl : ∀ {x : A} → x ≡ x

_≤_ : ℕ → ℕ → Bool
zero ≤ _ = true
suc _ ≤ zero = false
suc m ≤ suc n = m ≤ n

_m
¬ : Bool → Bool
¬ true = false
```

```
¬ false = true

_∧_ : Bool → Bool → Bool
true ∧ b = b
false ∧ _ = false

_∨_ : Bool → Bool → Bool
true ∨ _ = true
false ∨ b = b

ifZero : ℕ → Bool
ifZero zero = true
ifZero (suc _) = false

data _×_ (A B : Set) : Set where
_,_ : A → B → A × B

proj₁ : ∀ {A B : Set} → A × B → A
proj₁ (a , b) = a

proj₂ : ∀ {A B : Set} → A × B → B
proj₂ (a , b) = b

divideHelper : ℕ → ℕ → ℕ × ℕ
divideHelper zero m = zero , zero
divideHelper (suc n) m with n ... | true = zero , suc n
... | false = let qr = divideHelper n m in suc (proj₁ qr) , proj₂ qr

divides : ℕ → ℕ → Bool
divides m n with divideHelper n m
... | qr with ifZero (proj₂ qr)
... | true = true
... | false = false

hasProperFactor : ℕ → ℕ → Bool
hasProperFactor n zero = false
hasProperFactor n (suc zero) = false
hasProperFactor n (suc (suc m)) =
((suc (suc m)
isPrime : ℕ → Bool
isPrime n = (n > suc zero) ∧ ¬ (hasProperFactor n (n - suc zero))

data PrimeProof : Set where
mkPrimeProof : ∀ n → isPrime n ≡ true → PrimeProof

primeExample : PrimeProof
primeExample = mkPrimeProof (suc (suc zero)) refl
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda).
D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:79,16-17
Not in scope:
>
at D:\Daniel\Braude\Year 4\Semester G\FinalProject_Phase1\Proj\pythonProject\temp_proof.agda:79,16-17
when scope checking >
